

Desafio 1:

Sistema Web para Consulta e Armazenamento de Endereços

Objetivo:

Desenvolver um sistema web que permita ao usuário consultar um endereço pelo CEP, armazenar os registros consultados e exibir a lista de registros com opções de ordenação.

Requisitos Funcionais:

1. O usuário poderá digitar um CEP e buscar as informações do endereço através da API pública [ViaCEP](#).
2. Os dados retornados devem ser armazenados para consulta posterior. O formato de armazenamento pode ser escolhido pelo candidato (banco de dados, localStorage, JSON, etc.).
3. Deve haver uma interface para listar os endereços armazenados, permitindo que o usuário veja as informações salvas.
4. O usuário poderá ordenar a lista de endereços de forma crescente ou decrescente pelos seguintes campos:
 - Cidade
 - Bairro
 - Estado

Requisitos Técnicos:

- A busca do endereço deve ser realizada via **backend**, fazendo uma requisição HTTP para a API do [ViaCEP](#).
- A aplicação deve exibir mensagens de erro para CEPs inválidos ou falhas na requisição.
- A interface deve ser intuitiva, podendo ser desenvolvida com **HTML, CSS e JavaScript puro ou frameworks como React/Vue/Angular**.
- A ordenação dos registros deve ser implementada no backend.
- O candidato pode utilizar qualquer tecnologia para armazenar os dados (Ex: localStorage, IndexedDB, banco de dados relacional ou NoSQL).

Desafio 2:

Desafio: Implementação, Otimização e Comparação de Algoritmos de Ordenação

Objetivo:

Desenvolver três algoritmos de ordenação diferentes e realizar uma análise detalhada de desempenho, explorando fatores como tempo de execução e impacto em diferentes tipos de entrada.

Requisitos:

1. Desenvolva três funções para ordenar uma lista de números utilizando os seguintes algoritmos:
 - **QuickSort**
 - **MergeSort**
 - **BubbleSort**
2. Execute testes comparativos para os três algoritmos em diferentes cenários, incluindo:
 - **Listas pequenas (10 a 100 elementos)**
 - **Listas médias (1.000 a 10.000 elementos)**
 - **Listas grandes (100.000 elementos ou mais)**
 - **Listas já ordenadas** (caso melhor)
 - **Listas reversamente ordenadas** (caso pior)
 - **Listas com muitos elementos duplicados**
3. **Medição de Desempenho:**
 - Utilize ferramentas nativas da linguagem para medir **tempo de execução**
 - Compare o impacto dos diferentes algoritmos em listas de tamanhos variados.
4. **Análise de Complexidade e Eficiência:**
 - Explique a complexidade de tempo e espaço de cada algoritmo (Big O).
 - Compare os resultados obtidos e discuta quais algoritmos são mais apropriados para cada cenário.
 - Sugira otimizações adicionais para cada algoritmo e explique seus impactos.

Requisitos Técnicos:

- O código deve ser escrito em alguma linguagem backend de sua escolha.
- Deve seguir boas práticas de programação, incluindo modularização, uso adequado de estruturas de dados e documentação do código.
- O tempo de execução devem ser capturados e apresentados em formato de tabela ou gráfico para facilitar a análise.